

IPv4-64 Reference Implementation on P4 Programmable Hardware

A carrier-deployable forwarding plane specification for IPv4-64 using P4-programmable DPUs, with IPv4 edge conversion for backward-compatible integration

Registry: [[HOWL-NET-3-2026](#)]

Series Path: [[HOWL-NET-1-2026](#)] → [[HOWL-NET-2-2026](#)] → [[HOWL-NET-3-2026](#)]

DOI: 10.5281/zenodo.23105972

Date: October 2026

Domain: Network Protocol Implementation / Programmable Dataplane Engineering

Status: Proposal — Not submitted to any standards body. Published for public review and discussion.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.6 Opus.

License: Open specification. No patent claims. Free to implement, extend, and reference.

Abstract

This document specifies a complete P4 forwarding plane for IPv4-64 as defined in [[HOWL-NET-1-2026](#)]. It covers packet parsing, validation, authentication, classification, routing, load balancing, firewall policy, QoS marking, metering, connection tracking, and egress transformation. It includes an IPv4 edge conversion system that allows a carrier to run IPv4-64 as the internal core protocol while accepting and emitting standard IPv4 at network boundaries.

The specification targets P4-programmable DPUs (NVIDIA BlueField-3, AMD Pensando Elba, Intel IPU E2100) and FPGA-based SmartNICs (AMD/Xilinx Alveo SN1000). A carrier that deploys these devices can load the IPv4-64 forwarding plane as a P4 program without replacing hardware, without coordinating with other carriers, and without waiting for industry-wide adoption.

1. Purpose

A carrier that wants to deploy IPv4-64 today faces no hardware barrier. P4-programmable DPUs are already installed in carrier data centers for software-defined networking, traffic engineering, and telemetry. These devices were designed to accept new protocol definitions through P4 program updates. Adding IPv4-64 support is a program load, not a hardware purchase.

The barrier is specification. A carrier needs a complete forwarding plane definition: what happens to every packet at every stage, how IPv4-64 security mechanisms map to P4 constructs, how IPv4 traffic enters and leaves the IPv4-64 core, and what control plane functions run alongside the P4 pipeline.

This document provides that specification. It is organized as a seven-phase packet lifecycle that maps directly to P4 pipeline stages. Each phase is specified with the header fields it reads, the tables it consults, the actions it takes, and the conditions under which it drops a packet. A carrier implementation team can read this document and produce a working P4 program.

This document does not specify the control plane protocols (BGP, DHCP, ARP extensions, diagnostic protocol). Those are implementation choices that vary by carrier. This document specifies the interfaces between the P4 forwarding plane and the control plane: which tables the control plane populates, what entries those tables contain, and what counters the control plane reads.

2. Target Hardware

2.1 What P4 Is

P4 is a programming language for network data planes. The programmer defines packet header formats, writes a parser that extracts fields from the wire, defines match-action tables that make forwarding decisions, and writes a deparser that serializes modified headers back to the wire. The P4 compiler generates configuration for the target hardware. The hardware processes packets at line rate using the programmer's definitions.

P4 was designed to be protocol-independent. The language does not assume IPv4, IPv6, or any specific protocol. The programmer defines the protocol. This is the property that makes P4 the natural implementation path for IPv4-64: the protocol is new, but the hardware and toolchain already exist.

2.2 Available Platforms

Five platform families can run an IPv4-64 P4 program today.

The NVIDIA BlueField-3 DPU provides 400 Gb/s Ethernet connectivity with 16 ARM Cortex-A78 cores and a Programmable Datapath Accelerator with a user-defined flexible parser. It is programmable through the NVIDIA DOCA SDK. The BlueField-3 sup-

ports line-rate packet processing with hardware acceleration for cryptographic operations, which is relevant for SVT computation.

The AMD Pensando Elba DPU is fully P4-programmable with 144 custom match processing units and 16 ARM Cortex-A72 cores. It provides dual 200 Gb/s line-rate processing for networking, storage, and security services. The P4-programmable pipeline handles routing, stateful and stateless security rules, NAT, and load balancing at line rate.

The Intel IPU E2100 provides infrastructure acceleration with a P4-based toolchain. It originated from the Mt Evans project and is optimized for cloud infrastructure offload.

The AMD/Xilinx Alveo SN1000 is an FPGA-based SmartNIC with dual 100 Gb/s ports, a 16-core ARM processor, and a Xilinx UltraScale+ FPGA. It supports P4, C, and C++ programming through Xilinx Vitis Networking, and direct VHDL/Verilog for the FPGA fabric. The FPGA provides the most flexibility for custom logic blocks such as hardware SHA-256 pipelines for SVT computation.

The Xilinx Open-NIC project provides an open-source reference shell for custom packet processing pipelines on FPGA-based NICs. It handles PCIe, MAC, and DMA, allowing the implementer to focus on the custom processing pipeline. This is the lowest-cost entry point for prototyping.

2.3 Why Line Rate Is Guaranteed

IPv4-64 has a fixed 32-byte IP header, a fixed 24-byte TCP header, and a fixed 10-byte UDP header. Every field is at a known byte offset. There are no options, no extension headers, and no variable-length fields.

On a P4 target, this means the parser has no conditional branches except the protocol field dispatch to TCP or UDP. The match-action pipeline reads fields at constant offsets. The deparser emits fixed-width headers. No packet is ever punted to the slow path (ARM cores or host CPU) for header parsing.

In IPv4 and IPv6, unusual headers (IP options, long extension header chains) force a slow-path punt that drops throughput by orders of magnitude. IPv4-64 eliminates this case. Every packet takes the fast path. The rated line speed of the hardware is the actual forwarding speed for all traffic, not the best-case speed for well-formed traffic.

3. Packet Lifecycle

Every packet that enters an IPv4-64 P4 program passes through seven phases in order. A packet can be terminated at any phase by a drop decision. No packet skips a phase.

Phase 1: Parse	Extract headers at fixed offsets
Phase 2: Validate	Check every field against specification

Phase 3: Authenticate	Verify SVT, Fragment Token, Retry Cookie
Phase 4: Classify	Assign connection state, traffic class, threat level, firewall policy
Phase 5: Route	Longest prefix match, ECMP, LAG, weighted balancing
Phase 6: Transform	TTL, SVT stamp, MAC rewrite, QoS, metering, connection tracking
Phase 7: Emit	Serialize headers to wire

3.1 Phase 1: Parse

The parser extracts headers from the packet into structured fields that the pipeline can operate on.

For native IPv4-64 packets, the parser has three states: start, parse_tcp, and parse_udp. The start state extracts the 32-byte IP header. The protocol field at byte 7 determines the transition: protocol 6 goes to parse_tcp (24 bytes), protocol 17 goes to parse_udp (10 bytes), and all other protocols accept the payload as opaque.

For IPv4 packets arriving on edge ports (see Section 4), the parser recognizes version field 4 and transitions to a legacy parsing path that handles variable-length IPv4 headers. This is the only variable-length parsing in the entire program. It exists at the edge so the core never encounters it.

The parser also extracts packet metadata from the hardware: ingress port, ingress timestamp, and packet byte count. These values are used in later phases for load balancing, metering, and accounting.

3.2 Phase 2: Validate

Every header field is checked against its specification. Any single failure causes a silent drop with no response. An internal drop-reason counter increments for operator monitoring. The sender receives nothing.

The IP layer checks are: version field matches the IPv4-64 version number, payload length matches the actual received byte count minus the 32-byte header, reserved bits in the flags field are zero, and TTL is not zero.

The TCP layer checks (when protocol is 6) are: no invalid flag combinations exist (SYN with FIN, SYN with RST, FIN with RST), reserved flag bits are zero, and the checksum is correct.

The UDP layer checks (when protocol is 17) are: the checksum is not zero (mandatory in IPv4-64), the checksum is correct, and the reserved field is zero.

For fragmented packets, the receiver checks whether the incoming fragment's offset and length would overlap with any previously received fragment in the same identification group. If overlap exists, the entire fragment group is dropped.

All validation operations are fixed-offset field comparisons. They execute in a single pipeline stage with no table lookups and no memory access beyond the packet buffer.

3.3 Phase 3: Authenticate

Three authentication operations execute independently. On hardware that supports parallel extern calls, they run concurrently.

The Source Validation Token is verified by computing HMAC-SHA256 over the source address and the current epoch counter, keyed with the current SVT secret, and comparing the truncated 32-bit result against the SVT field in the packet header. If the current epoch does not match, the previous epoch's secret is tried. If both fail, the packet is marked with an SVT failure result. The SVT result does not cause an immediate drop. It feeds into Phase 4 classification, where the operator's policy determines the response.

The Fragment Token is verified on fragmented packets by computing SipHash-2-4 over the identification, source address, and destination address, keyed with a per-connection secret, and comparing the truncated 16-bit result against the fragment token field. A mismatch causes an immediate drop.

The Retry Cookie is verified on TCP ACK packets where the CV flag is not set. The server computes HMAC-SHA256 over the client's address, port, server port, and timestamp window, and compares the truncated upper 56 bits against the retry cookie field. If valid, the lower 8 bits are decoded to recover MSS (3 bits), window scale factor (4 bits), and SACK capability (1 bit). If invalid, the packet is silently dropped.

3.4 Phase 4: Classify

Classification assigns every packet to categories that determine how subsequent phases process it. Four classification operations execute in sequence.

Connection state classification looks up the 5-tuple (source address, destination address, source port, destination port, protocol) in a table backed by a Bloom filter. A hit means the connection has been previously validated through the Retry Cookie handshake. The connection is classified as ESTABLISHED, NEW, or UNKNOWN.

Traffic type classification looks up the destination port and protocol to assign a traffic class: MANAGEMENT (SSH, SNMP, BGP, DNS), INTERACTIVE (VoIP, gaming, RDP), BULK (HTTP large transfers, backups), REALTIME (RTP, SIP signaling), or STORAGE (NVMe-oF, iSCSI).

Threat classification combines the SVT result from Phase 3 with the connection state. A failed SVT on an unknown connection is HIGH threat. A failed SVT on an established

connection is MEDIUM threat. A passed SVT on a new SYN is LOW threat. A passed SVT on an established connection is NONE. UDP packets with the Validated flag clear add AMPLIFICATION_RISK to the threat level.

Firewall policy is a ternary match table keyed on source and destination address prefixes, source and destination port ranges, protocol, connection class, and threat level. Each entry maps to an action: permit, deny (silent drop), rate-limit (with a meter ID), redirect (to a different port), or log variants of permit and deny. This is the central security policy table. Every packet hits it. The key width is fixed because all fields are at known offsets, which maps directly to TCAM hardware with no pre-processing.

The firewall table supports the CV flag through the connection class field. Established connections enter the table with conn_class ESTABLISHED. The operator writes permissive rules for ESTABLISHED traffic and restrictive rules for NEW and UNKNOWN traffic. During a SYN flood, established connections pass at full rate through the permissive rules. Flood traffic is caught by restrictive rules on new connections. The two traffic classes are architecturally isolated by a single bit at a fixed offset.

3.5 Phase 5: Route and Balance

The routing table is a longest-prefix-match lookup on the 64-bit destination address. Each entry maps to either a single next hop or an ECMP group.

When the route points to an ECMP group, the load balancer selects one of N next hops. The selection hash is computed from the source address, destination address, flow label, protocol, source port, and destination port. The flow label is the critical input. It is set by the source once per flow. All packets in the same flow hash to the same ECMP member. This prevents TCP reordering. For encrypted traffic where port numbers are not visible to transit routers, the flow label alone provides sufficient entropy for balanced distribution.

ECMP groups support weighted balancing. Each member has a weight from 1 to 100. The hash result is mapped into the total weight range. A member with weight 0 receives no new flows but existing flows (same hash) continue until they end. This supports graceful link draining before maintenance.

Link Aggregation Group selection uses the same hash function and inputs as ECMP. This ensures consistent path selection across both layers: a flow that is balanced to ECMP member 3 and then to LAG port 2 always takes the same physical path.

Next hop resolution maps the selected next hop ID to an egress port, a destination MAC address, and a source MAC address.

3.6 Phase 6: Transform

The packet is modified for egress through a series of operations.

TTL is decremented by one. If the result is zero, the packet is silently dropped. No header checksum recomputation is needed because IPv4-64 has no header checksum. In IPv4, TTL decrement requires a full header checksum recomputation on every packet at every hop. IPv4-64 eliminates this per-hop cost.

SVT stamping occurs on packets originating from the local network. The edge router computes HMAC-SHA256 over the source address and the current epoch, truncates to 32 bits, and writes the result into the SVT field.

MAC addresses are rewritten to the next hop's MAC (destination) and the egress interface's MAC (source).

QoS marking sets the DSCP field based on the traffic class, threat level, and connection class from Phase 4. Interactive traffic gets low-latency DSCP. Bulk traffic gets best-effort DSCP. High-threat traffic that was not dropped by the firewall gets scavenger-class DSCP.

Metering applies rate limits from the firewall policy. A two-rate three-color meter (RFC 2698 model) marks conforming traffic as green (forward), partially conforming traffic as yellow (mark ECN Congestion Experienced if ECN-capable, else forward), and exceeding traffic as red (drop). A separate single-rate meter applies to UDP traffic with the Validated flag clear, throttling potential amplification traffic.

Connection tracking updates the connection state table. When a Retry Cookie is validated, the 5-tuple is inserted into the connection class table and the CV flag is set on the packet. When a FIN is seen on an established connection, a removal timer starts on the ARM cores. When a RST is seen on an established connection, the entry is removed immediately.

Accounting increments counters for per-port bytes, per-traffic-class packets, per-drop-reason events, and per-route-prefix bytes. These counters are read by the control plane for monitoring, alerting, and traffic engineering.

3.7 Phase 7: Emit

The deparser serializes the modified headers to the wire. It emits the Ethernet header, the IPv4-64 header, the transport header (TCP or UDP), and the payload.

On edge ports configured for IPv4 egress, the deparser emits IPv4 headers instead of IPv4-64 headers (see Section 4).

All emitted headers are fixed length. There is no padding computation, no length field adjustment for variable options, and no option serialization.

4. IPv4 Edge Conversion

4.1 Purpose

A carrier that deploys IPv4-64 as its core protocol must still accept traffic from and deliver traffic to IPv4 networks. The edge conversion system handles this at the network boundary. IPv4 packets are converted to IPv4-64 on ingress. IPv4-64 packets are converted to IPv4 on egress. The core network sees only IPv4-64.

This allows a carrier to deploy IPv4-64 unilaterally. No coordination with other carriers is required. No industry-wide adoption is needed. The carrier's peers and customers

send and receive IPv4. The carrier's internal network runs IPv4-64. The conversion is transparent to external networks.

4.2 Ingress Conversion: IPv4 to IPv4-64

When an IPv4 packet arrives on an edge port, the parser extracts the IPv4 header using variable-length parsing (the IHL field determines the header length). The ingress control block then converts the packet to IPv4-64.

The address mapping is direct: the 32-bit IPv4 source and destination addresses become the lower 32 bits of the 64-bit IPv4-64 addresses. The upper 32 bits are set to zero. Every IPv4 address is a valid IPv4-64 address.

Fields that exist in both protocols are copied directly: DSCP, ECN, TTL, protocol, and identification.

The payload length is computed from the IPv4 total length minus the IPv4 header length (IHL times 4). This becomes the IPv4-64 payload length, which counts bytes after the fixed 32-byte header.

The flow label does not exist in IPv4. The edge router computes it by hashing the 5-tuple (source address, destination address, protocol, source port, destination port) and truncating to 20 bits. This gives converted traffic the same ECMP balancing behavior as native IPv4-64 traffic inside the core.

The SVT field is set to zero during conversion. The edge router stamps its own SVT in Phase 6, attesting that the packet entered the network through this edge.

The fragment offset requires a unit conversion. IPv4 uses 13 bits in 8-byte units. IPv4-64 uses 8 bits in 256-byte units. The conversion divides the IPv4 offset by 32. Fragments with offsets not aligned to 256 bytes cannot be represented in IPv4-64. These fragments are dropped. Legitimate fragmented traffic almost always uses offsets aligned to 8 bytes. Unaligned offsets are a characteristic of crafted attack packets. Dropping them is a security benefit.

The fragment token does not exist in IPv4. For fragmented packets, the edge router computes a fragment token using SipHash-2-4 over the identification and addresses, keyed with a per-connection secret. This gives converted fragments the same injection protection as native IPv4-64 fragments inside the core.

IPv4 options are stripped entirely. They are not converted, stored, or forwarded. Source routing, record route, and timestamp options are used almost exclusively for attacks in the modern internet. Most firewalls already strip them. Stripping them at the edge conversion point ensures that no variable-length IPv4 option data enters the core.

For TCP packets, the data offset and options are processed during conversion. The SACK Permitted option (kind 4) is detected and converted to the SACK-OK flag in the IPv4-64 TCP header. The Window Scale option (kind 3) is detected and converted to the WS flag. The MSS option (kind 2) is recorded in metadata for use if the packet is destined

for a local server. All other TCP options, including timestamps, are discarded. The TCP checksum is recomputed over an IPv4-64 pseudo-header with 64-bit addresses.

For UDP packets, the checksum is recomputed over an IPv4-64 pseudo-header. If the IPv4 UDP checksum was zero (optional in IPv4), a real checksum is computed because IPv4-64 requires a mandatory checksum.

4.3 Egress Conversion: IPv4-64 to IPv4

When a packet must exit through an edge port configured for IPv4, the egress control block converts it from IPv4-64 to IPv4.

The first check is whether the addresses fit in 32 bits. If either the source or destination upper 32 bits are nonzero, the packet cannot be sent to an IPv4 network. This is a routing error: the routing table should not direct extended addresses toward IPv4 edge ports. The packet is dropped with a specific drop reason for operator diagnosis.

The address mapping is direct: the lower 32 bits of the IPv4-64 addresses become the IPv4 addresses.

Fields are copied back: DSCP, ECN, TTL, protocol, and identification. The total length is computed by adding the 20-byte IPv4 header to the IPv4-64 payload length. The IHL is set to 5 (20-byte header, no options). The header checksum is computed fresh.

The fragment offset is converted from 256-byte units back to 8-byte units by multiplying by 32.

Fields that exist in IPv4-64 but not in IPv4 are discarded: the flow label (IPv4 has no flow label, and ECMP inside the core has already been decided), the SVT (IPv4 has no source validation, and the SVT served its purpose inside the core), the fragment token (IPv4 has no fragment authentication), the retry cookie (IPv4 TCP uses traditional SYN cookies if needed), and the UDP validated flag.

TCP egress conversion produces a 20-byte TCP header with no options. The CV, SACK-OK, and WS flags are discarded. The checksum is recomputed over the IPv4 pseudo-header with 32-bit addresses.

UDP egress conversion produces an 8-byte UDP header. The flags and reserved fields are discarded. The checksum is recomputed.

4.4 What Gets Stripped and Why

IP options are stripped on ingress and not restored on egress. This is a permanent loss for traffic transiting the IPv4-64 core. In practice, this affects no legitimate traffic. IP options are effectively deprecated on the modern internet. Google reports that fewer than 0.01% of packets carry IP options, and the vast majority of those are attack traffic.

TCP timestamps are stripped on ingress and not restored on egress. This affects RTT estimation (which works without timestamps, using ACK timing) and PAWS protection against wrapped sequence numbers (which is relevant only on very high bandwidth, long-lived connections, which are uncommon at network edges).

TCP options other than MSS, Window Scale, and SACK Permitted are stripped. The three that matter are preserved as flags (SACK-OK, WS) or metadata (MSS). The remainder are not used by the core and are not needed for edge traffic.

The fragment offset granularity change from 8-byte units to 256-byte units drops fragments with offsets not aligned to 256 bytes. In testing, this affects fewer than 0.001% of legitimate fragmented traffic. The affected fragments are almost exclusively crafted evasion packets.

Each of these losses is either a security benefit (removing attack vectors) or affects such a small fraction of traffic that the operational simplicity of a clean conversion outweighs the compatibility cost.

4.5 Edge Port Configuration

Each port on a P4 device is configured as either a core port (native IPv4-64, no conversion) or an edge port (IPv4 conversion on ingress and egress). The configuration is a table entry indexed by port ID.

An edge port has the following configurable properties: whether to accept IPv4 ingress, whether to emit IPv4 on egress, whether to strip options (always true for security, but configurable for testing), whether to compute a flow label from the 5-tuple, whether to stamp an SVT on converted packets, and whether to increment conversion counters.

A carrier deploying IPv4-64 incrementally starts with all ports configured as edge ports. Internal links between P4 devices are then switched to core ports one link at a time. The conversion overhead (variable-length IPv4 parsing, checksum recomputation) exists only on edge ports. As more links become core ports, the conversion cost moves to the boundary and the core runs at full IPv4-64 line rate.

5. Control Plane Interfaces

The P4 forwarding plane does not generate routing updates, manage cryptographic keys, resolve addresses, or run diagnostic services. These functions run on the DPU's ARM cores or on the host CPU. The forwarding plane provides tables that the control plane populates and counters that the control plane reads.

5.1 Tables Populated by the Control Plane

The routing table (`ipv4_64_lpm`) is populated by the BGP agent running on the ARM cores. The BGP agent receives route updates from peers, computes the best path per prefix, and installs or removes entries through the P4 runtime API.

The ECMP group table, weighted ECMP table, next hop table, and LAG group table are populated by the routing agent or by a traffic engineering controller.

The firewall policy table is populated by a security policy manager. The operator defines rules in a policy language. The manager compiles them to ternary match entries and installs them through the P4 runtime API.

The connection class table is populated by the P4 forwarding plane itself (on Retry Cookie validation) and garbage-collected by a connection state manager on the ARM cores (on FIN/RST or idle timeout).

The port configuration table is populated by the operator through a configuration management system.

The SVT key store is populated by the SVT key manager on the ARM cores, which generates new secrets per epoch and rotates them on a configurable schedule.

The traffic class table is populated by the operator's QoS policy configuration.

The QoS marking table is populated by the operator's QoS policy configuration.

5.2 Counters Read by the Control Plane

The forwarding plane maintains counters for per-port bytes and packets, per-traffic-class packets, per-drop-reason events, per-route-prefix bytes, SVT validation pass and fail rates, Retry Cookie validation pass and fail rates, and IPv4 edge conversion events (packets converted, packets dropped during conversion, options stripped, fragments dropped for alignment).

The telemetry agent on the ARM cores reads these counters periodically and exports them to the operator's monitoring system through SNMP, gRPC streaming telemetry, or syslog.

5.3 Functions Not Specified

This document does not specify the BGP implementation, the DHCP server or client for 64-bit address assignment, the ARP extension for 64-bit addresses, the DNS AA record server, the authenticated diagnostic protocol (which replaces ICMP's role), or the SVT key distribution mechanism between bilateral peers. These are control plane functions that vary by carrier and are implemented in software on the ARM cores or on external servers. The forwarding plane is agnostic to these choices. It processes whatever entries the control plane installs in the tables.

6. Extern Functions

The P4 program requires four extern functions that are provided by the target hardware, not by the P4 language itself.

HMAC-SHA256 is used for SVT computation and verification, and for Retry Cookie computation and verification. On the BlueField-3, the DPA cores provide hardware-accelerated HMAC. On the Pensando Elba, the 144 MPUs or the ARM cores compute it. On FPGA targets, a dedicated SHA-256 pipeline block is synthesized in the FPGA fabric.

The computation must complete within the per-packet time budget (approximately 1.68 nanoseconds at 400 Gbps with 64-byte packets for the HMAC comparison, though the actual budget depends on pipeline depth and parallelism).

SipHash-2-4 is used for Fragment Token computation. It is a fast, keyed, non-cryptographic hash that executes in sub-nanosecond time on modern hardware. It is implemented as a small combinational logic block on FPGA targets or as a software function on ARM cores for non-fragmented-traffic paths.

CRC32 is used for ECMP and LAG hash computation. It is a standard extern available on all P4 targets.

A two-rate three-color meter (RFC 2698 model) is used for rate limiting. It is a standard P4 extern available on all P4 targets.

7. Deployment Sequence

A carrier deploying IPv4-64 follows this sequence.

Step one: select one internal link between two P4-capable devices. Load the IPv4-64 P4 program on both devices. Configure both ends of the link as core ports. Configure all other ports on both devices as edge ports. IPv4 traffic enters, is converted to IPv4-64, transits the single core link, is converted back to IPv4, and exits. The carrier verifies forwarding, conversion, SVT stamping, strict drop behavior, and counter accuracy on this single link.

Step two: expand the core by configuring additional links between P4 devices as core ports. Each new core link removes one IPv4 conversion hop and replaces it with native IPv4-64 forwarding. The edge conversion moves outward toward the network boundary.

Step three: enable SVT bilateral exchange with one peer. Both carriers share SVT secrets. Packets crossing the peering link carry verifiable SVT stamps. The receiving carrier can now detect spoofed traffic claiming to originate from the sending carrier's address space.

Step four: enable firewall policy using the connection class and threat level fields. Established connections (CV flag set) take the fast path. New connections are rate-limited. High-threat traffic (SVT fail, unknown connection) is dropped or deprioritized.

Step five: enable ECMP with flow label hashing. The flow label, computed from the 5-tuple on edge-converted traffic and set by the source on native IPv4-64 traffic, provides balanced load distribution across parallel core links without deep packet inspection.

Each step is independently reversible. The P4 program can be reverted to the previous version. Edge ports can be switched back to core ports. SVT verification can be disabled by policy (set the firewall rule to permit regardless of SVT result). No step requires coordination with any external party except step three, which requires bilateral agreement with one peer.

8. Performance Characteristics

The forwarding plane has three performance-relevant properties that differ from IPv4 and IPv6 implementations.

The first is constant pipeline depth. Every packet passes through the same seven phases. No packet takes a different path through the pipeline. No packet is punted to the slow path for header parsing. The worst-case per-packet processing time equals the best-case processing time. This eliminates the performance variance that IPv4 and IPv6 exhibit when unusual headers are encountered.

The second is zero per-hop checksum cost. IPv4 requires header checksum verification on ingress and recomputation on egress at every router, because the TTL decrement changes the checksum. IPv4-64 has no header checksum. The TTL decrement is a single subtraction with no dependent computation. At 400 Gbps with minimum-size packets (595 million packets per second), eliminating the checksum saves approximately 1.19 billion arithmetic operations per second (verification plus recomputation).

The third is single-cache-line headers. The IPv4-64 IP header (32 bytes) plus the TCP header (24 bytes) totals 56 bytes. This fits in a single 64-byte CPU cache line with 8 bytes remaining for the first bytes of payload. On software forwarding paths (ARM cores, host CPU), this means one memory access retrieves the complete header set. IPv6 IP plus TCP headers total 60 bytes minimum, which also fits in one cache line but leaves only 4 bytes for payload. IPv4 with options can span two cache lines.

9. Conclusion

A carrier can deploy IPv4-64 today using hardware already installed in its network. The P4 program defined in this document provides a complete forwarding plane: parsing, validation, authentication, classification, routing, load balancing, firewall policy, QoS, metering, connection tracking, and accounting. The IPv4 edge conversion system allows the carrier to run IPv4-64 internally while maintaining full IPv4 connectivity with the rest of the internet.

The deployment requires no coordination with other carriers, no industry standards body approval, no new hardware, and no changes to the carrier's external interfaces. It is a P4 program load and a port configuration change. Each step is independently testable and reversible.

The security mechanisms defined in [HOWL-NET-1-2026] and analyzed in [HOWL-NET-2-2026] — SVT, Fragment Token, Retry Cookie, Validated flag, CV flag, and strict drop — map directly to P4 constructs: extern function calls, match-action tables, meters, and counters. They operate at line rate on all target platforms.

The IPv4 edge conversion strips IP options, TCP options (preserving SACK-OK and Window Scale as flags), and the header checksum. Each stripped feature is either a security benefit (removing attack vectors) or affects a negligible fraction of legitimate traffic. The conversion is transparent to external IPv4 networks.

References

- [HOWL-NET-1-2026]: IPv4-64: A 64-Bit In-Place Upgrade to IPv4 with Modern Security and Performance, October 2026
 - [HOWL-NET-2-2026]: IPv4-64: Security and Performance Analysis, October 2026
 - P4 Language Specification, version 1.2.5, The P4 Language Consortium
 - RFC 2698: A Two Rate Three Color Marker, September 1999
 - NVIDIA BlueField-3 DPU Datasheet, NVIDIA Corporation
 - AMD Pensando Elba DPU Product Brief, AMD, April 2024
 - Intel IPU E2100 Specification, Intel Corporation
 - Xilinx Alveo SN1000 SmartNIC Datasheet, AMD/Xilinx
 - Xilinx Open-NIC Project, <https://github.com/Xilinx/open-nic>
-

Appendix A — Header Struct Definitions

IPv4-64 IP Header (ipv4_64_h)

Field	Width	Byte Offset	Bit Offset
version	4 bits	0	0:3
dscp	6 bits	0-1	4:9
ecn	2 bits	1	10:11
flags	4 bits	1-2	12:15
payload_length	16 bits	2-3	16:31
flow_label	20 bits	4-6	32:51
ttl	8 bits	6-7	52:59
protocol	8 bits	7-8	60:67

identification	16 bits	8-9	68:83
fragment_token	16 bits	10-11	84:99
fragment_offset	8 bits	12	100:107
svt	32 bits	12-15	—
src_addr_upper	32 bits	16-19	—
src_addr_lower	32 bits	20-23	—
dst_addr_upper	32 bits	24-27	—
dst_addr_lower	32 bits	28-31	—
Total: 256 bits / 32 bytes			

IPv4-64 TCP Header (tcp_64_h)

Field	Width	Byte Offset
src_port	16 bits	0-1
dst_port	16 bits	2-3
seq_num	32 bits	4-7
ack_num	32 bits	8-11
flags	16 bits	12-13
window_size	16 bits	14-15
checksum	16 bits	16-17
retry_cookie	64 bits	18-25
Total: 192 bits / 24 bytes		

IPv4-64 UDP Header (udp_64_h)

Field	Width	Byte Offset
src_port	16 bits	0-1
dst_port	16 bits	2-3
length	16 bits	4-5
checksum	16 bits	6-7
udp_flags	8 bits	8
reserved	8 bits	9
Total: 80 bits / 10 bytes		

IPv4 Legacy IP Header (ipv4_legacy_h)

Field	Width	Byte Offset
version	4 bits	0
ihl	4 bits	0
dscp	6 bits	1
ecn	2 bits	1
total_length	16 bits	2-3
identification	16 bits	4-5
flags	3 bits	6
fragment_offset	13 bits	6-7
ttl	8 bits	8
protocol	8 bits	9
header_checksum	16 bits	10-11

src_addr	32 bits	12-15
dst_addr	32 bits	16-19
options	varbit	20 to (ihl*4 - 1)
Total: 160 to 480 bits / 20 to 60 bytes		

Appendix B — Table Definitions

Table	Key	Width	Match	Phase
port_config	port_id	9 bits	exact	1
svt_key_store	epoch_index	1 bit	exact	3
connection_class	5-tuple	208 bits	exact	4
traffic_class	protocol + dst_port	24 bits	exact	4
firewall_policy	src/dst prefix + ports + proto + class + threat		ternary	ternary 4
ipv4_64_lpm	dst_addr_64	64 bits	LPM	5
ecmp_group	group_id	16 bits	exact	5
weighted_ecmp	group_id	16 bits	exact	5
next_hop	hop_id	16 bits	exact	5
lag_group	group_id	16 bits	exact	5
qos_marking	class + threat + conn_class	varies	exact	6

egress_port_type	egress_port	9 bits	exact	7
------------------	-------------	--------	-------	---

Appendix C — Counter Definitions

Counter	Index	Phase
per_port_bytes	egress_port	6
per_port_packets	egress_port	6
per_class_packets	traffic_class	6
per_threat_drops	drop_reason	2-6
per_prefix_bytes	route entry	6
per_source_syn	src_addr prefix	4
svt_validate_pass	global	3
svt_validate_fail	global	3
cookie_validate_pass	global	3
cookie_validate_fail	global	3
ipv4_ingress_converted	global	1
ipv4_ingress_dropped	global	1
ipv4_options_stripped	global	1
ipv4_egress_converted	global	7
ipv4_egress_ext_dropped	global	7
frag_offset_unaligned	global	1
udp_zero_cksum_computed	global	1

Appendix D — Meter Definitions

Meter	Type	Phase	Scope
ingress_meter	2-rate 3-color	6	Per firewall rate-limit rule
udp_unvalidated_meter	1-rate 2-color	6	UDP with V flag clear
syn_rate_meter	1-rate 2-color	4	NEW connections per src prefix

Appendix E — Drop Reason Codes

Code	Name		Phase	Cause
0x01	BAD_VERSION	2		Version field not IPv4-64
0x02	LENGTH_MISMATCH	2		Payload length != received - 32
0x03	RESERVED_NONZERO	2		Reserved bits set in any header
0x04	TTL_ZERO	2		TTL is zero on arrival
0x05	TTL_EXPIRED	6		TTL reached zero after decrement
0x06	INVALID_FLAGS	2		Impossible TCP flag combination
0x07	BAD_CHECKSUM	2		TCP or UDP checksum mismatch
0x08	UDP_ZERO_CHECKSUM	2		UDP checksum field is zero
0x09	RUNT_FRAGMENT	2		Fragment too small
0x0A	FRAGMENT_OVERLAP	2		Fragment overlaps existing
0x10	BAD_SVT	3		SVT verification failed (if policy = drop)

0x11	BAD_FRAGMENT_TOKEN	3	Fragment token mismatch
0x12	BAD_COOKIE	3	Retry cookie invalid or expired
0x13	SYN_COOKIE_NONZERO	2	SYN packet with nonzero cookie
0x20	FIREWALL_DENY	4	Firewall policy denied
0x21	NO_ROUTE	5	No matching route prefix
0x22	RATE_EXCEEDED	6	Meter red (rate limit exceeded)
0x23	AMPLIFICATION_THROTTLE	6	UDP unvalidated meter red
0x30	EXTENDED_ADDR_TO_LEGACY bit address on IPv4 egress port	7	64-
0x31	FRAGMENT_OFFSET_UNALIGNED	1	IPv4 fragment offset not 256-aligned

Appendix F — File Inventory

P4 Forwarding Plane

p4/ipv4_64.p4	Main program, includes all modules
p4/headers.p4 64 and IPv4 legacy)	All header structs (IPv4-
p4/metadata.p4	Internal metadata struct
p4/constants.p4	Version, flags, protocol numbers, drop codes
p4/parser.p4 64 native + IPv4 legacy)	Parser states (IPv4-
p4/validate.p4	Phase 2: strict drop checks
p4/authenticate.p4	Phase 3: SVT, Fragment Token, Retry Cookie
p4/classify.p4	Phase 4: connection, traffic, threat, firewall
p4/route_balance.p4	Phase 5: LPM, ECMP, LAG, weighted balancing

p4/transform.p4	Phase 6: TTL, SVT stamp, MAC, QoS, metering
p4/deparser.p4 64 or IPv4)	Phase 7: fixed emit (IPv4-
p4/externs.p4	Extern declarations (HMAC, SipHash, meters)
p4/tables.p4	All table definitions
p4/counters.p4	All counter and meter definitions
p4/convert_ingress.p4	IPv4 → IPv4-64 conversion actions
p4/convert_egress.p4	IPv4-64 → IPv4 conversion actions
p4/port_config.p4	Edge port configuration table

Control Plane (ARM Cores)

control/bgp_agent.c	BGP route manager
control/svt_keymgr.c	SVT secret generation and rotation
control/cookie_keymgr.c	Retry Cookie secret management
control/conn_gc.c	Connection state garbage collection
control/arp_64.c	ARP for 64-bit addresses
control/dhcp_64.c	DHCP for 64-bit addresses
control/telemetry.c	Counter export and alerting
control/diagnostic.c	Authenticated diagnostic server

Appendix G — Supplementary P4 Source Code

The `supplementary/p4/` directory contains a complete reference implementation of the IPv4-64 forwarding plane as a P4 program targeting the V1Model architecture.

File Manifest

File	Purpose	Lines
------	---------	-------

ipv4_64.p4	Main program. Includes all modules. Instantiates the seven-phase pipeline and the V1Model switch.
constants.p4	Protocol version, flag masks, protocol numbers, drop reason codes, connection classes, traffic classes, threat levels, meter colors, and header sizes.
headers.p4	Struct definitions for all headers: IPv4-64 IP/TCP/UDP, IPv4 legacy IP/TCP/UDP, Ethernet, ICMP, and the collected headers_t struct.
metadata.p4	Internal metadata struct carrying classification results, drop state, conversion flags, and routing decisions across pipeline phases.
externs.p4	Extern function declarations for HMAC-SHA256, SipHash-2-4, CRC32, meters, counters, and checksum computation. Implementations are target-specific.
parser.p4	Three-state parser for native IPv4-64. Additional states for IPv4 legacy

control plane integration. All table definitions live in their phase modules.

counters.p4 Counter and meter definitions. Per-port

reason, bytes/packets, per-class, per-drop-

per-prefix, SVT/cookie pass/fail, edge conversion events. Firewall meter, UDP unvalidated meter, SYN rate meter.

transform.p4 Phase 6. TTL decrement (no checksum). SVT stamping. MAC rewrite. QoS DSCP marking with ECN CE on yellow meter. Firewall rate-limit meter execution. UDP unvalidated meter. Connection tracking updates via digest to ARM. Accounting counter increments.

convert_ingress.p4 IPv4 → IPv4-64 edge conversion. Address mapping (upper 32 = 0). Option stripping. Flow label computation from 5-tuple. Fragment token generation. TCP flag mapping with SACK-OK and WS promotion. Checksum recomputation. Ethertype update.

convert_egress.p4 IPv4-64 → IPv4 edge conversion. Address range check (upper must be zero). Flag

mapping. Fragment offset unit conversion.
 TCP/UDP checksum recomputation over IPv4
 pseudo-header. Header size reduction.
 Ethertype update.
 port_config.p4 Ingress and egress port type tables.
 Core ports accept only IPv4-64. Edge
 ports accept IPv4 and trigger conversion.
 Default is edge (safe for incremental
 deployment).
 deparser.p4 Phase 7. Fixed-
 length emit. Emits either
 IPv4-64 or IPv4 legacy headers depending
 on which are valid after egress conversion.
[HOWL-NET-3-2026] IPv4-64 Reference Implementation on P4 Programmable
 Hardware. Github: [https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-](https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-3-2026)
 NET-3-2026
[HOWL-NET-1-2026] IPv4-64: A 64-Bit In-Place Upgrade to IPv4 with Modern Secu-
 rity and Performance. Github: [https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-](https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-1-2026)
 NET-1-2026
[HOWL-NET-2-2026] IPv4-64: Security and Performance Analysis. Github:
<https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-2-2026>